

Kubernetes & Percona PostgreSQL

Top 50 Alerts Runbook

Production Troubleshooting & Remediation Guide

SS&C Private Cloud — Database Reliability

How to Use This Runbook

1. Look up the alert in the Quick Index by name or alert ID.
2. Confirm the trigger condition (PromQL) and current symptoms.
3. Run the Diagnostic Commands section to gather evidence.
4. Follow Resolution Steps. Stop as soon as service is restored.
5. Review Prevention to harden against recurrence.

Severity Levels

CRITICAL — service-impacting; act immediately

WARNING — degraded; investigate and remediate within SLA

1. Environment Overview

This runbook covers two SS&C Private Cloud Kubernetes clusters that host production Percona PostgreSQL Operator deployments. Both clusters share a common monitoring stack and CNI but differ in cluster role and the naming convention used for the PerconaPGCluster custom resources.

Cluster Inventory (Observed)

Item	Cluster A (pgkubcluster)	Cluster B (salt-control-postgres)
Control-plane nodes	3 (etcd HA)	5 (etcd HA)
CNI	Cilium (Hubble enabled)	Cilium (Hubble enabled)
CSI driver	csi-sscloud	csi-sscloud
Monitoring	kube-prometheus-stack	kube-prometheus-stack
PG Operator	percona-postgresql-operator	percona-postgresql-operator
PG topology	i1 (primary set, 5/5), i2 (4/5)	i1 (5/5), i2 (5/5), i3 (5/5)
PgBouncer	2/2	0 (replicaset 0/0)
PMM	1/1	1/1
pgBackRest repo host	1/1 Running	1/1 Running
Backup CronJob	50 23 * * * (pgbackupmonitor)	50 23 * * * (pgbackupmonitor)
Cluster age	~401 days	~91 days

Recurring Alerts Currently Observed

Based on the email alert digest from Alertmanager (subjects beginning with **FIRING:**), the following alerts have been seen most often across both clusters and form the seed for this runbook:

- etcd cluster member communication is slow
- etcd cluster members are down
- DaemonSet rollout is stuck
- DaemonSet pods are misscheduled
- StatefulSet has not matched the expected number of replicas
- StatefulSet update has not been rolled out
- Job failed to complete (pgbackupmonitor)
- Node is not ready
- Node is unreachable
- Host is running out of memory
- System saturated, load per core is very high
- Memory major page faults are occurring at very high rate
- One or more targets are unreachable
- postgres backups failed

Severity Levels

Severity	Meaning	Response Time
CRITICAL	Service down, data at risk, RPO/RTO impact	Immediate (page on-call)
WARNING	Degraded service, capacity at risk	Within 1 hour
INFO	Awareness only, no immediate action	Next business day

2. Quick Index — All 50 Alerts

Alerts are grouped by category. Each row shows the alert ID, name, and severity. Use this as a fast lookup before diving into the detailed sections that follow.

etcd & Control Plane

ID	Alert	Severity
01	etcd cluster members are down	Critical
02	etcd cluster member communication is slow	Warning
03	etcd has high number of leader changes	Warning
04	etcd has insufficient members	Critical
05	etcd no leader	Critical
06	KubeAPIDown / kube-apiserver unreachable	Critical
07	KubeControllerManagerDown	Critical
08	KubeSchedulerDown	Critical

Node & Host

ID	Alert	Severity
09	Node is not ready (KubeNodeNotReady)	Warning
10	Node is unreachable (KubeNodeUnreachable)	Warning
11	Host is running out of memory	Warning
12	System saturated, load per core is very high	Warning
13	Memory major page faults are occurring at very high rate	Warning
14	Node disk is filling up (NodeDiskRunningFull)	Warning
15	NodeFilesystemAlmostOutOfFiles (inode exhaustion)	Warning
16	NodeNetworkReceiveErrs (network errors)	Warning

Workloads

ID	Alert	Severity
17	DaemonSet rollout is stuck	Warning
18	DaemonSet pods are misscheduled	Warning
19	StatefulSet has not matched the expected number of replicas	Warning
20	StatefulSet update has not been rolled out	Warning
21	KubeDeploymentReplicasMismatch	Warning
22	KubePodCrashLooping	Warning
23	KubePodNotReady	Warning
24	KubeContainerWaiting (ImagePullBackOff)	Warning

Pods, Jobs, Storage

ID	Alert	Severity
25	Job failed to complete	Warning
26	CronJob failed to schedule	Warning

27	KubePodOOMKilled	Warning
28	KubePersistentVolumeFillingUp	Warning
29	KubePersistentVolumeErrors	Critical
30	VolumeAttachment stuck	Warning

Monitoring Stack

ID	Alert	Severity
31	One or more targets are unreachable	Warning
32	Prometheus target absent (job_name absent)	Critical
33	PrometheusOperatorReconcileErrors	Warning
34	AlertmanagerConfigInconsistent	Critical

PostgreSQL / Percona

ID	Alert	Severity
35	postgres backups failed	Critical
36	PostgreSQLReplicationLag	Warning
37	PostgreSQLConnectionsTooHigh	Warning
38	PostgreSQLDeadlocks	Warning
39	PostgreSQL down / not responding	Critical
40	PostgreSQL Slow Queries / High Latency	Warning
41	PgBouncer connections high / pool saturated	Warning
42	PgBouncer Down	Critical
43	Percona Operator reconcile failed	Warning
44	Last successful backup is older than RPO	Critical

Network & Kubelet

ID	Alert	Severity
45	KubeAPILatencyHigh	Warning
46	Kubernetes Client Certificate Expiration	Warning
47	Cilium agent down on node	Critical
48	CoreDNS Down or High Error Rate	Warning
49	KubeletDown	Critical
50	KubeProxyDown	Critical

3. Detailed Troubleshooting Guides

The following sections present each of the 50 alerts in full detail. Each entry covers description, trigger condition, symptoms, root causes, diagnostic commands, step-by-step resolution, and preventive measures.

3.1 etcd & Control Plane

This category contains 8 alert(s). Each provides full diagnostic and remediation guidance.

ALERT #01

CRITICAL

etcd & Control Plane

etcd cluster members are down

Description

One or more etcd members are not reachable, threatening cluster quorum and consistency of cluster state.

Trigger Condition (PromQL)

```
max without (endpoint) (sum without (instance) (up{job=~".*etcd.*"} == bool 0)) > 0
```

Symptoms

- kubectl commands intermittently fail or hang
- API server returns 'etcdserver: request timed out' errors
- Pod scheduling stalls; new pods stuck in Pending
- Resource updates (kubectl apply) fail with leader election errors

Probable Root Causes

- etcd pod crashed or OOMKilled on a control-plane node
- Network partition between control-plane nodes
- Disk I/O saturation on etcd data directory (>10ms fsync latency)
- Control-plane node lost (hardware/VM failure)
- etcd database corruption or quorum loss after multiple node failures
- Certificate expiration on peer/client certs

Diagnostic Commands

```
# Check etcd pod status across all control-plane nodes
kubectl -n kube-system get pods -l component=etcd -o wide

# Check member health from inside an etcd pod
kubectl -n kube-system exec -it etcd-<control-plane-node> -- sh -c \
'ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
endpoint health --cluster -w table'

# List all etcd members and their status
kubectl -n kube-system exec -it etcd-<node> -- sh -c \
'ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
member list -w table'

# Check etcd logs for the failing member
kubectl -n kube-system logs etcd-<node> --tail=200

# Verify peer connectivity
kubectl -n kube-system exec etcd-<node> -- sh -c \
'ETCDCTL_API=3 etcdctl --endpoints=<peer-url>:2379 endpoint status'
```


Resolution Steps

1. Identify the failed member from 'member list' output (Status=unreachable).
2. SSH to the affected control-plane node and check kubelet/static pod manifests:
`sudo systemctl status kubelet`
`sudo ls -la /etc/kubernetes/manifests/etcd.yaml`
3. Check disk space on etcd data directory:
`sudo du -sh /var/lib/etcd`
`df -h /var/lib/etcd`
4. If etcd pod is crash-looping, check its logs and resource limits:
`sudo crictl logs $(sudo crictl ps -a --name etcd -q | head -1)`
5. If quorum is intact (majority alive), remove the failed member:
`etcdctl member remove <member-id>`
6. Re-add as a new member after fixing the underlying issue:
`etcdctl member add <name> --peer-urls=https://<ip>:2380`
7. If quorum is LOST, perform restore from snapshot:
`etcdctl snapshot restore <snapshot.db> --data-dir=/var/lib/etcd-restore`
8. Run defrag on healthy members to reclaim space:
`etcdctl defrag --cluster`

Prevention

- Take regular etcd snapshots (every 30 min) and store off-cluster
- Monitor etcd_disk_wal_fsync_duration_seconds (P99 should be <10ms)
- Use SSD/NVMe for etcd data directory; never co-locate with high-IO workloads
- Maintain odd number of control-plane nodes (3 or 5) for proper quorum
- Set etcd resource requests/limits explicitly; never rely on defaults
- Renew etcd certificates before expiry: kubeadm certs check-expiration

ALERT #02

WARNING

etcd & Control Plane

etcd cluster member communication is slow

Description

Round-trip time between etcd peers exceeds 150ms at the 99th percentile, indicating network or load issues that can lead to leader elections and write latency.

Trigger Condition (PromQL)

```
histogram_quantile(0.99, rate(etcd_network_peer_round_trip_time_seconds_bucket[5m])) > 0.15
```

Symptoms

- Slow kubectl operations, especially writes
- Leader election storms in etcd logs
- Increased API server request latency (>1s)
- PostgreSQL operator reconciliation takes longer than usual

Probable Root Causes

- Network congestion or packet loss between control-plane nodes
- Cross-AZ/cross-DC etcd cluster (high baseline latency)
- CPU starvation on etcd peer (noisy neighbor)
- Disk fsync latency on a peer (slow followers)
- MTU mismatch causing fragmentation
- Cilium agent issues on the node (eBPF program reload)

Diagnostic Commands

```
# Check peer round-trip metrics
kubectl -n monitoring port-forward svc/kube-prometheus-stack-prometheus 9090:9090
# Then query: histogram_quantile(0.99, rate(etcd_network_peer_round_trip_time_seconds_bucket[5m]))

# Direct ping test between control-plane nodes
ssh <cp-node-1> ping -c 50 <cp-node-2>

# Check WAL fsync latency
kubectl -n kube-system logs etcd-<node> | grep -i 'slow fdatsync'

# Check load average and CPU steal
ssh <cp-node> top -bn1 | head -5
ssh <cp-node> mpstat -P ALL 1 5
```

Resolution Steps

```
1. Identify the slow peer using etcd metrics by member ID.
2. Verify network path: traceroute, MTR between peers for 60+ seconds.
3. Check disk IOPS: iostat -xm 1 on the slow peer; etcd needs <10ms fsync.
4. If disk-bound, migrate etcd data dir to faster storage tier.
5. If CPU-bound, check for noisy neighbors and increase node size if needed.
6. Run defrag during a low-traffic window:
  etcdctl defrag --cluster --command-timeout=120s
7. Compact old revisions:
  rev=$(etcdctl endpoint status --write-out=fields | grep Revision | head -1 | awk '{print $3}')
  etcdctl compact $rev
```

Prevention

- Keep all etcd peers on the same low-latency network (<5ms RTT)
 - Monitor `etcd_disk_backend_commit_duration_seconds` and `wal_fsync_duration`
 - Schedule periodic compaction and defrag (`auto-compaction-mode=periodic`)
 - Set `heartbeat-interval=100ms` and `election-timeout=1000ms` appropriately
 - Avoid running other I/O-intensive pods on control-plane nodes (taint them)
-

ALERT #03

WARNING

etcd & Control Plane

etcd has high number of leader changes

Description

etcd has changed leaders 4 or more times within 15 minutes, indicating instability that affects API server availability.

Trigger Condition (PromQL)

```
increase((max without (instance) (etcd_server_leader_changes_seen_total) or 0*absent(etcd_server_leader_changes_seen_total))[15m:1m]) >= 4
```

Symptoms

- Sporadic 'etcdserver: leader changed' errors in API server logs
- Brief API outages every few minutes
- Kubernetes controller-manager and scheduler reconnecting frequently

Probable Root Causes

- Slow disk causing leader to lose heartbeats
- Network instability between peers
- Aggressive compaction blocking leader
- CPU throttling on the leader node

Diagnostic Commands

```
# Count leader changes in last hour
# Prometheus query: increase(etcd_server_leader_changes_seen_total[1h])

# Check current leader
kubectl -n kube-system exec etcd-<node> -- sh -c \
'ETCDCTL_API=3 etcdctl endpoint status --cluster -w table \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key'

# Inspect logs for 'lost leader' or 'elected leader' messages
kubectl -n kube-system logs etcd-<node> --since=1h | grep -E 'elected|lost'
```

Resolution Steps

1. Identify which member is repeatedly losing leadership.
2. Check that node's disk fsync latency and CPU steal time.
3. If a node is consistently flapping leadership, drain it and replace.
4. Increase election-timeout (default 1000ms) if cross-AZ:
--election-timeout=2500 in /etc/kubernetes/manifests/etcd.yaml
5. Reduce snapshot frequency if it's blocking the leader:
--snapshot-count=10000 (default 100000 is fine for most)

Prevention

- Pin etcd to dedicated nodes with no resource competition
- Use SSD with consistent IO performance (avoid burst credit storage)
- Keep compaction frequent to reduce DB size
- Alert on leader changes >= 3 per hour as early warning

ALERT #04

CRITICAL

etcd & Control Plane

etcd has insufficient members

Description

Less than majority of etcd members are reachable. Cluster cannot accept writes.

Trigger Condition (PromQL)

```
sum without (instance) (up{job=~".*etcd.*"} == bool 1) < ((count without (instance) (up{job=~".*etcd.*"}) + 1) / 2)
```

Symptoms

- All kubectl write operations fail
- Cluster effectively read-only or fully down
- Pods cannot be created, modified, or deleted

Probable Root Causes

- Multiple control-plane nodes failed simultaneously
- Network partition isolating etcd peers
- Storage backend (SAN/PV) outage

Diagnostic Commands

```
# This is a P0 incident; verify quorum status
kubectl -n kube-system get pods -l component=etcd

# From any reachable etcd member
etcdctl --endpoints=https://127.0.0.1:2379 member list
etcdctl endpoint health --cluster
```

Resolution Steps

1. Declare incident; cluster is in a degraded state.
2. Identify which members are alive **and** which are dead.
3. Restore failed member nodes **if** they are recoverable (boot, network).
4. **If** majority cannot be restored, proceed with disaster recovery:
 - a. Locate latest **etcd** snapshot (should be in **/apps/backups/etcd/**).
 - b. **Stop** kubelet **on** remaining nodes.
 - c. Restore the snapshot to a single node:

```
etcdctl snapshot restore /apps/backups/etcd/snapshot.db \
  --name=<node-name> \
  --initial-cluster=<node-name>=https://<ip>:2380 \
  --initial-advertise-peer-urls=https://<ip>:2380 \
  --data-dir=/var/lib/etcd-restore
```
 - d. **Update etcd** manifest to point to restored data-dir.
 - e. **Start etcd as** a single-node cluster.
 - f. Add other members back one-by-one with **'etcdctl member add'**.

Prevention

- Run 3 or 5 control-plane nodes spread across failure domains
- Automate etcd snapshots every 30 minutes
- Test snapshot restore procedure quarterly
- Document and rehearse cluster DR runbook

ALERT #05

CRITICAL

etcd & Control Plane

etcd no leader

Description

An etcd member has no leader. The cluster is not making progress.

Trigger Condition (PromQL)

```
etcd_server_has_leader == 0
```

Symptoms

- All cluster writes blocked
- API server logs show 'context deadline exceeded'

Probable Root Causes

- Quorum lost
- Election storm in progress
- Network split-brain

Diagnostic Commands

```
# Check which members report no leader
etcdctl endpoint status --cluster -w table

# Look for 'no leader' in member logs
kubectl -n kube-system logs etcd-<node> | grep -i 'leader'
```

Resolution Steps

1. Verify network connectivity between all peers.
2. If a single member shows no leader but others are healthy, **restart** that member.
3. If all members **show** no leader, treat **as** quorum loss (see Alert #4).
4. Check **for** clock skew between nodes (NTP sync):
chronyc tracking # on each control-plane node

Prevention

- Ensure NTP/chrony is running and synced on all control-plane nodes
- Use stable network with redundant paths
- Avoid graceful etcd restarts during peak write load

ALERT #06

CRITICAL

etcd & Control Plane

KubeAPIDown / kube-apiserver unreachable

Description

kube-apiserver has been unreachable from Prometheus for >15 minutes.

Trigger Condition (PromQL)

```
absent(up{job="apiserver"} == 1)
```

Symptoms

- kubectl commands fail with connection refused or timeout
- Webhooks and admission controllers fail
- Operator (Percona PostgreSQL Operator) cannot reconcile

Probable Root Causes

- kube-apiserver pod crash on all control-plane nodes
- etcd unavailable (apiserver depends on it)
- Certificate expiration
- LoadBalancer in front of API misconfigured

Diagnostic Commands

```
# Check apiserver pod status
kubectl -n kube-system get pods -l component=kube-apiserver -o wide

# If kubectl fails, SSH to a control-plane node
sudo crictl ps -a --name kube-apiserver
sudo crictl logs <kube-apiserver-container-id> --tail=200

# Check certificate expiration
sudo kubeadm certs check-expiration

# Verify health endpoint locally
curl -k https://127.0.0.1:6443/livez?verbose
```

Resolution Steps

1. SSH directly to a control-plane node when kubectl is down.
2. Verify kubelet is running: `systemctl status kubelet`
3. Check static pod manifest: `ls /etc/kubernetes/manifests/kube-apiserver.yaml`
4. If certs expired, renew with: `sudo kubeadm certs renew all`
5. Restart kubelet to pick up renewed certs: `systemctl restart kubelet`
6. If etcd is down, fix etcd first (see Alerts 1-5).
7. If LB unreachable, verify VIP and backend health checks.

Prevention

- Run apiserver in HA mode (3+ instances)
- Monitor cert expiration 30 days in advance
- Auto-renew certs via cronjob: `kubeadm certs renew all`
- Maintain LB health checks against /livez and /readyz

ALERT #07

CRITICAL

etcd & Control Plane

KubeControllerManagerDown

Description

kube-controller-manager has disappeared from Prometheus target discovery.

Trigger Condition (PromQL)

```
absent(up{job="kube-controller-manager"} == 1)
```

Symptoms

- Deployments do not roll out
- Endpoints do not get updated when pods change
- PVCs are not bound to PVs
- Garbage collection of orphaned objects stops

Probable Root Causes

- Controller-manager pod crash on all control-plane nodes
- Leader election failure (etcd issue)
- Misconfigured kubeconfig in static manifest

Diagnostic Commands

```
kubectl -n kube-system get pods -l component=kube-controller-manager
kubectl -n kube-system logs kube-controller-manager-<node> --tail=200

# Check who holds the leader lease
kubectl -n kube-system get lease kube-controller-manager -o yaml
```

Resolution Steps

1. Verify the static pod manifest is unchanged:
/etc/kubernetes/manifests/kube-controller-manager.yaml
2. Check kubelet logs for any pod start failures
3. Restart kubelet on the affected node
4. If lease is stuck, delete it to force re-election:
kubectl -n kube-system delete lease kube-controller-manager

Prevention

- Run 3 control-plane nodes for leader-election redundancy
- Monitor leader election via 'leader_election_master_status' metric
- Alert on KubeControllerManagerDown for >5min

ALERT #08

CRITICAL

etcd & Control Plane

KubeSchedulerDown

Description

kube-scheduler is not reporting metrics; pod scheduling will be impacted.

Trigger Condition (PromQL)

```
absent(up{job="kube-scheduler"} == 1)
```

Symptoms

- New pods stay in Pending state
- Failover of statefulsets stalls
- PostgreSQL pod replacement does not progress

Probable Root Causes

- Scheduler pod crash on all control-plane nodes
- Leader election failure
- Custom scheduler profile misconfiguration

Diagnostic Commands

```
kubectl -n kube-system get pods -l component=kube-scheduler
kubectl -n kube-system logs kube-scheduler-<node> --tail=200
kubectl get pods -A --field-selector=status.phase=Pending
```

Resolution Steps

1. Inspect scheduler logs for panics or config errors.
2. Verify static pod manifest at `/etc/kubernetes/manifests/kube-scheduler.yaml`.
3. Restart kubelet on each control-plane node sequentially.
4. Check leader lease: `kubectl -n kube-system get lease kube-scheduler -o yaml`

Prevention

- Run scheduler in HA across 3 control-plane nodes
- Validate custom scheduler profiles in non-prod first
- Pin scheduler image version; do not use :latest

3.2 Node & Host

This category contains 8 alert(s). Each provides full diagnostic and remediation guidance.

ALERT #09

WARNING

Node & Host

Node is not ready (KubeNodeNotReady)

Description

A node has been in NotReady state for more than 15 minutes. Pods on it become unreachable and are eventually evicted.

Trigger Condition (PromQL)

```
kube_node_status_condition{condition="Ready",status="true"} == 0
```

Symptoms

- `kubectl get nodes` shows NotReady
- Pods on the node show 'Unknown' status after 5 minutes
- Workloads experience reduced capacity / failed scheduling
- PostgreSQL replica may need to fail over if its node is affected

Probable Root Causes

- kubelet stopped or crashed on the node
- Container runtime (containerd/CRI-O) hung or restarted
- Disk pressure (kubelet eviction threshold reached)
- Memory pressure
- PID pressure (kernel.pid_max exhausted)
- Network unreachable (node isolated)
- CNI plugin (Cilium) failure

Diagnostic Commands

```
# Inspect node conditions
kubectl describe node <node-name> | sed -n '/Conditions:/,/Addresses:/p'

# Check kubelet status on the node
ssh <node> sudo systemctl status kubelet
ssh <node> sudo journalctl -u kubelet --since '15 min ago' --no-pager | tail -100

# Check container runtime
ssh <node> sudo systemctl status containerd
ssh <node> sudo crictl info | head -20

# Check Cilium agent on the node
kubectl -n kube-system get pods -l k8s-app=cilium -o wide --field-selector
spec.nodeName=<node>

# Check disk and memory pressure
ssh <node> df -h /var/lib/kubelet /var/lib/containerd
ssh <node> free -m
ssh <node> cat /proc/loadavg
```

Resolution Steps

```
1. SSH to the node and check kubelet logs for the root error.
2. If kubelet is down: sudo systemctl restart kubelet
3. If containerd hung: sudo systemctl restart containerd; then restart kubelet
4. If disk pressure: clean up unused images: sudo crictl rmi --prune
5. If memory pressure: identify top consumers and drain other pods.
6. If Cilium-related: kubectl -n kube-system delete pod -l k8s-app=cilium
--field-selector spec.nodeName=<node>
7. If unrecoverable, cordon and drain, then replace the node:
  kubectl cordon <node>
  kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
8. For Percona PostgreSQL pods, check if failover happened:
  kubectl -n postgres-operator get pg <cluster-name>
  kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- patronictl
list
```

Prevention

- Set kubelet eviction-soft thresholds with grace periods
 - Monitor disk usage on /var/lib/kubelet and /var/lib/containerd
 - Use image garbage collection: --image-gc-high-threshold=85
 - Configure CPU/Memory reservations: --system-reserved, --kube-reserved
 - Run node-problem-detector for kernel-level issue detection
-

ALERT #10

WARNING

Node & Host

Node is unreachable (KubeNodeUnreachable)

Description

Node has been tainted as unreachable, meaning the control plane cannot communicate with kubelet on this node.

Trigger Condition (PromQL)

```
(kube_node_spec_taint{key="node.kubernetes.io/unreachable",effect="NoExecute"}) == 1
```

Symptoms

- Node shows NotReady,SchedulingDisabled
- Pods get the node.kubernetes.io/unreachable:NoExecute taint
- Pods evicted after toleration period (default 5 min)

Probable Root Causes

- Network failure between node and control-plane
- Node fully powered off or VM crashed
- Firewall change blocking kubelet port (10250)

Diagnostic Commands

```
# Try direct ping/SSH to the node
ping <node-ip>
ssh <node>

# Test kubelet API from a control-plane node
curl -k https://<node-ip>:10250/healthz

# From the node (if reachable), verify outbound to apiserver
ssh <node> 'curl -k https://<apiserver-vip>:6443/livez'
```

Resolution Steps

1. If node is alive but isolated, fix network (check routes, firewall, security group).
2. If kubelet is bound but rejecting (cert issue), check `/var/lib/kubelet/pki/`.
3. If node is unrecoverable, force-delete its objects:
`kubectl delete node <node-name>`
then provision a replacement and rejoin via `kubeadm join`
4. Verify Percona Postgres replicas auto-recovered after eviction:
`kubectl -n postgres-operator get pods -l postgres-operator.crunchydata.com/instance`

Prevention

- Use redundant network paths (bonded NICs, multiple uplinks)
- Audit firewall rules whenever security policies change
- Maintain spare capacity to absorb a node failure
- Ensure podAntiAffinity for HA workloads (Patroni replicas already do this)

ALERT #11

WARNING

Node & Host

Host is running out of memory

Description

Node has less than 10% available memory. Risk of OOMKilled pods and kernel OOM killer firing.

Trigger Condition (PromQL)

```
(node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100 < 10
```

Symptoms

- Pods getting OOMKilled (Exit code 137)
- Node enters MemoryPressure condition
- PostgreSQL processes terminated by kernel
- kswapd consuming high CPU

Probable Root Causes

- Pod memory requests/limits not set, allowing overcommit
- Memory leak in an application (e.g., Grafana, Prometheus, PG processes)
- Sudden surge in workload (large query, big batch job)
- Page cache pressure from heavy I/O
- PostgreSQL shared_buffers set too high relative to node memory

Diagnostic Commands

```
# Top memory consumers on the node
ssh <node> 'ps auxf --sort=-%mem | head -20'

# Pod-level memory usage
kubectl top pods -A --sort-by=memory | head -20

# Check if any PG pod is high
kubectl -n postgres-operator top pods

# Cgroup memory pressure
ssh <node> 'cat /sys/fs/cgroup/memory.pressure'

# Check OOM kill events
ssh <node> 'sudo dmesg -T | grep -i oom | tail -20'
kubectl get events -A --field-selector reason=OOMKilling
```

Resolution Steps

1. Identify the **top** consumer (pod **or** system process).
2. If a Postgres pod, check active queries **and** connection count:
`kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- \`
`psql -c "SELECT pid, username, state, query FROM pg_stat_activity ORDER BY`
`backend_start;"`
3. Cancel runaway query: `SELECT pg_cancel_backend(<pid>);`
4. Drain non-critical pods **from** the node:
`kubectl drain <node> --ignore-daemonsets --pod-selector=...`
5. If a memory leak in app pod, **restart** that pod:
`kubectl -n <ns> delete pod <pod>`
6. **Right-size** requests/limits:
`kubectl -n postgres-operator edit perconapgcluster <cluster>`
7. Add **more** memory to the node **or** scale horizontally.

Prevention

- Always set memory requests AND limits on every pod
 - Set Percona PG pod memory limit at 1.5x of `shared_buffers` + `work_mem`*`max_connections`
 - Enable cgroup v2 with `memory.high` for graceful pressure
 - Monitor `working_set_bytes` vs request to detect bloat
 - Run kube-prometheus-stack memory-limit alerts at 80% and 90%
-

ALERT #12

WARNING

Node & Host

System saturated, load per core is very high

Description

5-minute load average per CPU core exceeds 1.5, indicating sustained CPU saturation.

Trigger Condition (PromQL)

```
node_load5 / count without (cpu) (node_cpu_seconds_total{mode="idle"}) > 1.5
```

Symptoms

- Slow response from all workloads on the node
- PostgreSQL query latency increases
- kubelet pod sync slows down
- Patroni health checks may flap

Probable Root Causes

- Runaway query consuming all CPU
- Vacuum/Autovacuum on a large table
- pg_repack or pg_dump running concurrently
- Container without CPU limits hogging the node
- High context switch rate (too many connections)

Diagnostic Commands

```
# CPU and load summary
ssh <node> 'uptime && top -bn1 | head -15'

# Top CPU consumers
ssh <node> 'ps auxf --sort=-%cpu | head -10'

# Per-CPU breakdown
ssh <node> 'mpstat -P ALL 1 5'

# In Postgres, find expensive queries
kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- \
psql -c "SELECT pid, now()-query_start AS duration, state, query \
FROM pg_stat_activity WHERE state='active' \
ORDER BY duration DESC LIMIT 10;"

# Check pg_stat_statements top consumers
kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- \
psql -c "SELECT query, calls, mean_exec_time, total_exec_time \
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;"
```

Resolution Steps

1. Cancel the highest-cost queries: `SELECT pg_cancel_backend(<pid>);`
2. Throttle autovacuum if it's the cause: `ALTER SYSTEM SET autovacuum_vacuum_cost_limit = 1000;`
3. Drain non-critical pods to off-load the node.
4. Add CPU limits to misbehaving pods.
5. Add nodes if this is steady-state load, not a spike.

Prevention

- Set CPU requests and limits on every pod
- Use HPA on stateless workloads to auto-scale

- Tune Postgres connection pool (PgBouncer transaction mode)
 - Schedule heavy maintenance (REINDEX, VACUUM FULL) during off-hours
 - Enable pg_stat_statements; review weekly
-

ALERT #13

WARNING

Node & Host

Memory major page faults are occurring at very high rate

Description

Node is experiencing >500 major page faults per second, indicating heavy disk-backed memory access (swap or memory-mapped I/O).

Trigger Condition (PromQL)

```
rate(node_vmstat_pgmajfault[5m]) > 500
```

Symptoms

- Very high I/O wait
- Slow Postgres queries with normal CPU usage
- Database response time spikes

Probable Root Causes

- Working set larger than physical RAM (cache misses)
- Swap being used heavily (despite swap normally being disabled on K8s nodes)
- Cold-start: large query reading data not yet in page cache
- Postgres shared_buffers + work_mem causes paging

Diagnostic Commands

```
# Check vmstat for page faults and IO
ssh <node> 'vmstat 1 10'

# Verify swap is off (it should be on K8s nodes)
ssh <node> 'swapon --show && cat /proc/swaps'

# Postgres buffer cache hit ratio
kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- \
  psql -c "SELECT sum(blks_hit)*100.0/sum(blks_hit+blks_read) AS cache_hit_ratio \
    FROM pg_stat_database;"
```

Resolution Steps

1. Confirm swap is off on the node. If on, disable: `swapoff -a` + remove from `fstab`.
2. Check Postgres `effective_cache_size`; should be 50-75% of node memory.
3. Increase `shared_buffers` if cache hit ratio < 95%.
4. Look at large sequential scans and add indexes:
`SELECT * FROM pg_stat_user_tables ORDER BY seq_tup_read DESC LIMIT 10;`
5. Move heavy reporting workloads to a replica.

Prevention

- Disable swap on all Kubernetes nodes
- Tune `shared_buffers`, `effective_cache_size` for the working set size
- Use SSD for Postgres data (already in `/apps/pgsql_data/<version>`)
- Pre-warm the cache after restart with `pg_prewarm` extension

ALERT #14

WARNING

Node & Host

Node disk is filling up (NodeDiskRunningFull)

Description

Filesystem on the node has less than 15% free space; risk of node entering DiskPressure.

Trigger Condition (PromQL)

```
(node_filesystem_avail_bytes{mountpoint="/" } * 100) /
node_filesystem_size_bytes{mountpoint="/" } < 15
```

Symptoms

- Node tainted with node.kubernetes.io/disk-pressure
- Pods evicted from node
- Container image pulls fail
- Postgres WAL writes may stall if /apps partition is full

Probable Root Causes

- Container logs not rotated
- Old container images not garbage collected
- PV expansion not done as data grew
- WAL files accumulating (archive_command failing)
- core dumps in /var/lib/systemd/coredump
- Local volume usage exceeded

Diagnostic Commands

```
# Disk usage by mountpoint
ssh <node> 'df -h'

# Largest directories
ssh <node> 'sudo du -h --max-depth=1 / 2>/dev/null | sort -rh | head -20'
ssh <node> 'sudo du -h --max-depth=1 /var/lib 2>/dev/null | sort -rh | head -20'

# Container log sizes
ssh <node> 'sudo du -sh /var/log/pods/* | sort -rh | head -20'

# Postgres data and WAL
kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- \
  bash -c 'du -sh /pgdata/pg*/pg_wal /pgdata/pg*/base'
```

Resolution Steps

1. Prune unused images: `ssh <node> 'sudo crictl rmi --prune'`
2. Prune unused container content: `ssh <node> 'sudo journalctl --vacuum-time=2d'`
3. Truncate large pod logs (do not delete; logs must rotate):
`ssh <node> 'sudo truncate -s 0 /var/log/pods/<bad-pod>/<container>.log'`
4. If Postgres pg_wal is full, fix archive_command/pgBackRest first:
`kubectl -n postgres-operator logs pgkubcluster-repo-host-0 -c pgbackrest`
5. Expand the underlying PVC if data legitimately grew:
`kubectl -n postgres-operator edit pvc <pvc-name> # increase storage`
6. Resize the filesystem after PV expansion (if not auto):
`kubectl exec -it <pod> -- resize2fs /dev/<device>`

Prevention

- Configure container log rotation: `kubelet --container-log-max-size=10Mi --container-log-max-files=5`
- Run image GC: `--image-gc-high-threshold=85`

- Monitor every PVC at 75% with warning, 90% with critical
 - Monitor pgBackRest archive backlog (archive_status_count)
 - Use pgbackrest archive-push async mode to avoid blocking
-

ALERT #15

WARNING

Node & Host

NodeFilesystemAlmostOutOfFiles (inode exhaustion)

Description

Less than 10% of inodes are free on a filesystem.

Trigger Condition (PromQL)

```
(node_filesystem_files_free{mountpoint="/" } * 100) /  
node_filesystem_files{mountpoint="/" } < 10
```

Symptoms

- Cannot create new files even though df shows free space
- Pod creation fails with 'no space left on device'

Probable Root Causes

- Many small files (e.g., session files, log files)
- Postgres temp files not cleaned
- Old image layers with many small files

Diagnostic Commands

```
ssh <node> 'df -i'  
  
# Count files per directory  
ssh <node> 'sudo find / -xdev -type f 2>/dev/null | head -10000 | awk -F/ \  
    "{ for (i=1;i<NF;i++) print \"$i }" | sort | uniq -c | sort -rn | head -20'
```

Resolution Steps

1. Identify the offending directory.
2. Clean temporary files: `find /tmp -mtime +7 -delete` (carefully).
3. Prune containerd: `crictl rmi --prune`
4. Reformat with more inodes if structurally needed (rare): `mkfs.ext4 -N <count>`

Prevention

- Use XFS for /var/lib/containerd (better inode behavior)
- Rotate logs aggressively
- Avoid millions-of-small-files patterns in apps; use object storage instead

ALERT #16

WARNING

Node & Host

NodeNetworkReceiveErrs (network errors)

Description

Network interface error rate above 1% of packets received over 2 minutes.

Trigger Condition (PromQL)

```
rate(node_network_receive_errs_total[2m]) / rate(node_network_receive_packets_total[2m]) > 0.01
```

Symptoms

- Slow or dropped pod-to-pod connections
- Patroni replication lag
- Cilium agent log errors

Probable Root Causes

- Bad cable or NIC
- MTU mismatch (especially with overlay networking)
- Driver bug
- Switch port flapping

Diagnostic Commands

```
ssh <node> 'ip -s link show'
ssh <node> 'ethtool -S <interface> | grep -i err'
ssh <node> 'dmesg | grep -i <interface>'

# Test MTU with don't-fragment ping
ssh <node> 'ping -M do -s 1472 <peer-ip>'
```

Resolution Steps

1. Check physical layer: cable, NIC LEDs, switch port counters.
2. Verify MTU consistency across nodes and CNI:
`kubectrl -n kube-system describe ds cilium | grep -i mtu`
3. Update NIC driver/firmware if hardware OK.
4. If interface is failed, switch to backup NIC in bond.
5. Drain and replace node if NIC is permanently bad.

Prevention

- Use bonded interfaces with LACP
- Match MTU end-to-end (including switches and CNI)
- Monitor ethtool counters via node_exporter textfile collector
- Keep NIC firmware/drivers patched

3.3 Workloads

This category contains 8 alert(s). Each provides full diagnostic and remediation guidance.

ALERT #17

WARNING

Workloads

DaemonSet rollout is stuck

Description

DaemonSet rollout has not progressed in 5 minutes. Affected DaemonSets in your clusters: cilium, csi-sscloud-node, kube-proxy, kube-prometheus-stack-prometheus-node-exporter.

Trigger Condition (PromQL)

```
((kube_daemonset_status_current_number_scheduled != kube_daemonset_status_desired_number_scheduled) or (kube_daemonset_status_number_misscheduled != 0) or (kube_daemonset_status_updated_number_scheduled != kube_daemonset_status_desired_number_scheduled) or (kube_daemonset_status_number_available != kube_daemonset_status_desired_number_scheduled)) and (changes(kube_daemonset_status_updated_number_scheduled[5m]) == 0)
```

Symptoms

- Updated pods don't appear on all nodes
- Some nodes still on old version
- Cilium/CSI/node-exporter inconsistency across nodes

Probable Root Causes

- New pod failing to start (image pull, init container error)
- Resource (CPU/Mem) requests not satisfiable on node
- Node taint preventing scheduling and DS doesn't tolerate
- PodDisruptionBudget blocking eviction of old pod
- RollingUpdate maxUnavailable too low

Diagnostic Commands

```
# Identify the stuck DaemonSet
kubectl get ds -A

# Detail
kubectl -n <ns> describe ds <name>

# Check pods and their nodes
kubectl -n <ns> get pods -l <ds-label> -o wide

# Look at the failing pod
kubectl -n <ns> describe pod <stuck-pod>
kubectl -n <ns> logs <stuck-pod> --previous

# Check rollout status
kubectl -n <ns> rollout status ds/<name>
kubectl -n <ns> rollout history ds/<name>
```


Resolution Steps

1. Inspect events **on** the failing pod **for** the specific cause.
2. ImagePullBackOff: verify image name **and** registry credentials
`kubectl -n <ns> get pods <pod> -o yaml | grep image:`
3. CrashLoopBackOff: check logs **from** previous container
`kubectl -n <ns> logs <pod> --previous --all-containers`
4. Insufficient resources: lower requests **or** add capacity to node.
5. Toleration missing **for** tainted node: add tolerations:
`kubectl -n <ns> patch ds <name> --type='json' -p='[{"op": "add", ...}]'`
6. Roll back **if** the new version is broken:
`kubectl -n <ns> rollout undo ds/<name>`

Prevention

- Test DS upgrades in dev/beta first
 - Use canary tolerations: deploy to 1 node first
 - Use updateStrategy.rollingUpdate.maxUnavailable thoughtfully (1 or 10%)
 - Always pin to a specific image tag (no :latest)
 - Validate manifests in CI with kubeconform / kubeval
-

ALERT #18

WARNING

Workloads

DaemonSet pods are misscheduled

Description

Pods of a DaemonSet are running on nodes that should not have them (selector mismatch).

Trigger Condition (PromQL)

```
kube_daemonset_status_number_misscheduled{job="kube-state-metrics",namespace=~".*"} > 0
```

Symptoms

- kube-state-metrics shows misscheduled count > 0
- Unexpected DS pods on a control-plane or specialty node

Probable Root Causes

- Node label changed after DS pod placed
- DaemonSet nodeSelector / nodeAffinity recently modified
- Node taint removed mid-flight

Diagnostic Commands

```
kubectl -n <ns> describe ds <name> | grep -A5 -i 'selector|tolerations'  
kubectl get nodes --show-labels | grep <selector-key>  
kubectl -n <ns> get pods -l <ds-label> -o wide
```

Resolution Steps

1. Decide whether the DS or the node label is the source of truth.
2. Re-label the node, or update the DS selector to match reality.
3. Delete the misscheduled pods to let the controller re-evaluate:
`kubectl -n <ns> delete pod <misscheduled-pod>`

Prevention

- Treat node labels as critical config; manage via IaC
- Avoid changing labels on running production nodes during business hours
- Use admission webhooks to validate DS selectors against existing labels

ALERT #19

WARNING

Workloads

StatefulSet has not matched the expected number of replicas

Description

A StatefulSet has fewer ready replicas than desired and hasn't progressed in 10 minutes. From your screenshots: pgkubcluster-i2-d6hm shows 4/5 (one pod degraded) and 0/1 in the second cluster.

Trigger Condition (PromQL)

```
(kube_statefulset_status_replicas_ready{job="kube-state-metrics"} !=  
kube_statefulset_status_replicas{job="kube-state-metrics"}) and  
(changes(kube_statefulset_status_replicas_updated{job="kube-state-metrics"}[10m]) == 0)
```

Symptoms

- Postgres replica missing from cluster (Patroni list shows fewer members)
- Reduced read capacity
- Risk to HA if primary fails

Probable Root Causes

- Pod stuck in Pending due to PVC binding failure
- Pod stuck in CrashLoopBackOff
- Node hosting the pod is NotReady
- PV provisioner (csi-sscloud) not creating volume
- podAntiAffinity prevents scheduling (e.g., need new node)
- Patroni cannot bootstrap from primary (timeline mismatch)

Diagnostic Commands

```
# StatefulSet status  
kubectl -n postgres-operator get sts pgkubcluster-i2-d6hm -o wide  
kubectl -n postgres-operator describe sts pgkubcluster-i2-d6hm  
  
# Failing pod details  
kubectl -n postgres-operator get pods -l postgres-operator.crunchydata.com/instance-set=i2  
kubectl -n postgres-operator describe pod pgkubcluster-i2-d6hm-0  
  
# PVC status  
kubectl -n postgres-operator get pvc | grep i2  
  
# Patroni state  
kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- patronictl list  
  
# Operator logs  
kubectl -n postgres-operator logs deploy/percona-postgresql-operator --tail=200
```

Resolution Steps

1. Check pod events: `kubectl -n postgres-operator describe pod <pod>`
2. PVC stuck: check storageclass and CSI driver pods.
`kubectl -n kube-system get pods -l app=csi-sscloud-controller`
`kubectl -n kube-system logs -l app=csi-sscloud-controller --tail=200`
3. Pod CrashLoopBackOff: get database logs.
`kubectl -n postgres-operator logs <pod> -c database --previous --tail=200`
4. Patroni timeline mismatch: re-init replica.
`kubectl -n postgres-operator exec <pod> -c database -- patronictl reinit <cluster> <member>`
5. AntiAffinity issue: scale node pool or change affinity rule.
6. If WAL gap from primary, may need pgBackRest restore.

Prevention

- Set sensible PVC requests; don't request more than storage class supports
 - Monitor PVC binding latency
 - Keep Percona Operator and PG image versions consistent across clusters
 - Maintain spare node capacity for replacements
 - Backup pgBackRest stanza before any major upgrade
-

ALERT #20

WARNING

Workloads

StatefulSet update has not been rolled out

Description

A StatefulSet is mid-update but new revision pods are not all created.

Trigger Condition (PromQL)

```
max without (revision) (kube_statefulset_status_current_revision unless kube_statefulset_status_update_revision) * (kube_statefulset_replicas != kube_statefulset_status_replicas_updated)
```

Symptoms

- Old and new pods coexist beyond expected window
- Postgres minor-version upgrade stuck

Probable Root Causes

- Update strategy is OnDelete and someone forgot to delete pods
- New revision pod failing readiness probe
- Slow PV detach/attach during pod replace

Diagnostic Commands

```
kubectl -n <ns> rollout status sts/<name>
kubectl -n <ns> rollout history sts/<name>
kubectl -n <ns> get pods -l <selector> -o jsonpath='{range .items[*]}{.metadata.name}{\t}{.spec.containers[*].image}{\n"}}{end}'
```

Resolution Steps

1. If updateStrategy=OnDelete, delete each old-revision pod one at a time and wait for ready:
kubectl -n <ns> delete pod <old-pod>
2. If RollingUpdate is stuck, fix the new pod's readiness/liveness probes.
3. Roll back if the new version is broken:
kubectl -n <ns> rollout undo sts/<name>

Prevention

- Test minor PG upgrades on non-prod first
- Verify pgBackRest backups before any sts upgrade
- Use Percona's documented upgrade process; do not hand-edit images

ALERT #21

WARNING

Workloads

KubeDeploymentReplicasMismatch

Description

Deployment available replicas differ from desired for >15 minutes.

Trigger Condition (PromQL)

```
(kube_deployment_spec_replicas{job="kube-state-metrics"} !=  
kube_deployment_status_replicas_available{job="kube-state-metrics"}) and  
(changes(kube_deployment_status_replicas_updated[10m]) == 0)
```

Symptoms

- App availability degraded
- From your env: pgbouncer at 2/2 expected; check if dropped
- Grafana / Prometheus operator unavailable

Probable Root Causes

- Pod scheduling failure
- Image pull failure
- Health check failures
- ConfigMap/Secret missing

Diagnostic Commands

```
kubectl -n <ns> describe deploy <name>  
kubectl -n <ns> get rs -l <selector>  
kubectl -n <ns> describe rs <newest-rs>  
kubectl -n <ns> get events --sort-by='.lastTimestamp' | tail -30
```

Resolution Steps

1. Identify the failing replicaset.
2. Inspect failing pod events and logs.
3. Common fixes:
 - ImagePullBackOff: fix image tag or pull secret
 - CrashLoopBackOff: fix command/args/env
 - Pending: add tolerations or scale node pool
4. If completely broken, roll back: `kubectl -n <ns> rollout undo deploy/<name>`

Prevention

- Always specify image with sha256 digest or pinned tag
- Use readiness probe + minReadySeconds for safer rollouts
- Use PodDisruptionBudget to keep at least N replicas available

ALERT #22

WARNING

Workloads

KubePodCrashLooping

Description

A container in a pod is restarting repeatedly.

Trigger Condition (PromQL)

```
max_over_time(kube_pod_container_status_waiting_reason{reason="CrashLoopBackOff",  
job="kube-state-metrics"}[5m]) >= 1
```

Symptoms

- Pod RESTARTS counter increasing
- App or database intermittently unavailable

Probable Root Causes

- Application config error
- Missing dependency (DB, secret, configmap)
- Liveness probe too aggressive
- OOMKilled (then restart, OOM, restart...)

Diagnostic Commands

```
kubectl -n <ns> logs <pod> --previous --tail=200  
kubectl -n <ns> describe pod <pod> | tail -30  
kubectl -n <ns> get events --field-selector involvedObject.name=<pod>
```

Resolution Steps

1. Read --previous logs for the actual crash reason.
2. If OOMKilled, raise memory limit or fix leak.
3. If config error, fix CM/Secret and bounce the pod.
4. If liveness probe failing too quickly, raise initialDelaySeconds and timeoutSeconds.

Prevention

- Always set both liveness and readiness probes (with sane delays)
- Use startupProbe for slow-starting databases
- Right-size memory limits with 30% headroom
- Validate configs in CI

ALERT #23

WARNING

Workloads

KubePodNotReady

Description

Pod has been in non-Ready state for more than 15 minutes.

Trigger Condition (PromQL)

```
sum by(namespace, pod, cluster) (max by(namespace, pod, cluster)
(kube_pod_status_phase{job="kube-state-metrics", phase!="Pending|Unknown|Failed"}) *
on(namespace, pod, cluster) group_left(owner_kind) topk by(namespace, pod, cluster) (1,
max by(namespace, pod, owner_kind, cluster) (kube_pod_owner{owner_kind!="Job"}))) > 0
```

Symptoms

- Service endpoints incomplete
- Reduced replica count for the app

Probable Root Causes

- Stuck in ContainerCreating (CSI/PVC issue)
- Stuck in Pending (no schedulable node)
- Init container failing

Diagnostic Commands

```
kubectl -n <ns> describe pod <pod>
kubectl -n <ns> logs <pod> -c <init-container>
kubectl get events -n <ns> --field-selector involvedObject.name=<pod>
```

Resolution Steps

1. Check pod events first; the symptom is usually visible there.
2. PVC issue → see Alert #29 (PVC errors).
3. Scheduling issue → check node taints, affinity, resources.
4. Init container failure → fix init script or required dependency.

Prevention

- Use validating admission webhooks to catch typos early
- Set timeoutSeconds appropriately on probes
- Maintain capacity headroom on node pools

ALERT #24

WARNING

Workloads

KubeContainerWaiting (ImagePullBackOff)

Description

Container has been Waiting for >1h. Most common reason is ImagePullBackOff.

Trigger Condition (PromQL)

```
sum by (namespace, pod, container, cluster) (kube_pod_container_status_waiting_reason{reason!="CrashLoopBackOff", job="kube-state-metrics"}) > 0
```

Symptoms

- Pod stays in O/N Ready
- Events show: 'Failed to pull image'

Probable Root Causes

- Wrong image name or tag
- Registry unreachable from node
- imagePullSecret missing or expired
- Image was deleted from registry
- Node disk full → cannot extract image

Diagnostic Commands

```
kubectl -n <ns> describe pod <pod> | grep -A5 Events

# Try pulling manually from a node
ssh <node> 'sudo crictl pull <image>'
```

Resolution Steps

1. Verify the image exists: `docker pull / crictl pull` from a node
2. Check imagePullSecrets in the pod's serviceaccount
`kubectl -n <ns> get sa default -o yaml`
3. Renew expired registry creds:
`kubectl -n <ns> create secret docker-registry regcred ... --dry-run=client -o yaml | kubectl apply -f -`
4. Free disk on node if extraction failed (Alert #14).

Prevention

- Use immutable image tags or digest references
- Monitor registry credential expiration
- Mirror critical images to internal registry

3.4 Pods, Jobs, Storage

This category contains 6 alert(s). Each provides full diagnostic and remediation guidance.

ALERT #25

WARNING

Pods, Jobs, Storage

Job failed to complete

Description

A Job did not succeed within its activeDeadlineSeconds or backoffLimit. Your env shows pgbackupmonitor-29518550, 29519990, 29521430 as Failed.

Trigger Condition (PromQL)

```
kube_job_failed{job="kube-state-metrics"} > 0
```

Symptoms

- Failed Job objects accumulating
- Backup monitoring data not collected for those windows

Probable Root Causes

- pgBackRest verify failure
- Network issue when accessing repo-host
- Permissions on /apps/backups
- Resource limits caused OOM
- Bug in custom job logic

Diagnostic Commands

```
# Check failed jobs
kubectl -n postgres-operator get jobs --field-selector status.successful=0

# Get the failing pod from the job
kubectl -n postgres-operator get pods --selector=job-name=<job-name>

# Pod logs
kubectl -n postgres-operator logs <pod>

# pgBackRest stanza-level info
kubectl -n postgres-operator exec pgkubcluster-repo-host-0 -- pgbackrest info
```

Resolution Steps

1. Read the failed pod's logs to identify root cause.
2. If pgBackRest related, run a manual stanza check:
`kubectl -n postgres-operator exec pgkubcluster-repo-host-0 -- pgbackrest check`
3. If repo connectivity, verify repo-host pod is running:
`kubectl -n postgres-operator get pods -l postgres-operator.crunchydata.com/pgbackrest`
`=`
4. Manually run a backup to confirm fix:
`kubectl annotate -n postgres-operator pg pgkubcluster`
`postgres-operator.crunchydata.com/pgbackrest-backup="$(date)" --overwrite`
5. Clean up failed Jobs:
`kubectl -n postgres-operator delete job <failed-job>`

Prevention

- Set ttlSecondsAfterFinished to auto-clean old jobs
- Alert on failed jobs immediately, not after they pile up
- Test backup restore monthly to confirm backups are usable
- Monitor pgBackRest archive-push and check pgBackRest log:
`kubectl -n postgres-operator exec pgkubcluster-repo-host-0 -- tail -f /pgbackrest/log/db-backup.log`

ALERT #26

WARNING

Pods, Jobs, Storage

CronJob failed to schedule

Description

CronJob has not run in 24+ hours though it should have. Affects pgkubcluster-pgbackupmonitor (50 23 * * *).

Trigger Condition (PromQL)

```
kube_cronjob_status_active{job="kube-state-metrics"} == 0 and (time() - kube_cronjob_status_last_schedule_time{job="kube-state-metrics"}) > (60 * 60 * 24)
```

Symptoms

- Last schedule time stale
- Backup monitoring data missing for last day

Probable Root Causes

- Cronjob suspended (suspend=true)
- Concurrency policy = Forbid and previous still running
- Time zone misconfiguration
- Controller-manager down at scheduled time

Diagnostic Commands

```
kubectl -n postgres-operator get cj
kubectl -n postgres-operator describe cj pgkubcluster-pgbackupmonitor

# Check controller-manager logs around the scheduled time
kubectl -n kube-system logs kube-controller-manager-<node> | grep -i cronjob
```

Resolution Steps

1. Confirm SUSPEND=False in CronJob `status`.
2. Verify timezone: `kubectl get cj <name> -o jsonpath='{.spec.timeZone}'`
3. Manually trigger a job:
`kubectl -n postgres-operator create job --from=cronjob/pgkubcluster-pgbackupmonitor manual-$(date +%s)`
4. If concurrencyPolicy=Forbid, kill stuck previous job.

Prevention

- Use successfulJobsHistoryLimit and failedJobsHistoryLimit to control bloat
- Set startingDeadlineSeconds to skip missed runs gracefully
- Monitor cronjob_last_schedule_time vs schedule

ALERT #27

WARNING

Pods, Jobs, Storage

KubePodOOMKilled

Description

A container was killed by the kernel OOM killer.

Trigger Condition (PromQL)

```
increase(kube_pod_container_status_terminated_reason{reason="OOMKilled"}[5m]) > 0
```

Symptoms

- Pod RESTARTS increasing
- Reason field in 'kubectl describe' shows OOMKilled
- App memory grows then dies

Probable Root Causes

- Memory limit too low
- Memory leak in app
- Sudden workload spike (large query)
- JVM Xmx too high relative to container limit

Diagnostic Commands

```
kubectl get pods -A -o json | jq '.items[] | \
  select(.status.containerStatuses[]?.lastState.terminated.reason=="OOMKilled") | \
  {ns: .metadata.namespace, pod: .metadata.name}'

kubectl describe pod <oomkilled-pod> | grep -A3 'Last State'

# Memory usage over time
# Prometheus: container_memory_working_set_bytes{pod="<pod>"}
```

Resolution Steps

1. Inspect memory usage trend leading to `kill` (`working_set` vs `limit`).
2. Increase the `limit` if usage is legitimate.
3. If leak suspected, capture heap dump (for JVM) or `ps_mem`.
4. For Postgres: check `shared_buffers + work_mem * max_connections` vs `limit`.
`shared_buffers + (work_mem * max_connections) + (temp_buffers * max_connections)`
should be `<` container memory `limit`.

Prevention

- Set request and limit to roughly equal values for predictable scheduling
- Profile memory usage in non-prod before promoting
- Use VPA recommendations as a starting point

ALERT #28

WARNING

Pods, Jobs, Storage

KubePersistentVolumeFillingUp

Description

PV has less than 10% free and is predicted to fill up within 4 days.

Trigger Condition (PromQL)

```
(kubelet_volume_stats_available_bytes{job="kubelet"} /
kubelet_volume_stats_capacity_bytes{job="kubelet"}) < 0.10 and
predict_linear(kubelet_volume_stats_available_bytes[6h], 4 * 24 * 3600) < 0
```

Symptoms

- Postgres write errors when full
- WAL accumulating fast
- Backups consuming repo PV

Probable Root Causes

- Database growth not anticipated
- WAL retention policy too long
- Backup retention misconfigured
- Bloat in tables/indexes

Diagnostic Commands

```
kubectl get pvc -A -o custom-columns=NS:.metadata.namespace,NAME:.metadata.name,CAPACITY:.status.capacity.storage

# Inside the Postgres pod
kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- df -h /pgdata
kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- du -sh /pgdata/pg*/pg_wal /pgdata/pg*/base

# Check biggest tables
kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- psql -c \
  "SELECT relname, pg_size_pretty(pg_total_relation_size(relid)) AS size \
  FROM pg_stat_user_tables ORDER BY pg_total_relation_size(relid) DESC LIMIT 10;"

# pgBackRest repo size
kubectl -n postgres-operator exec pgkubcluster-repo-host-0 -- du -sh /pgbackrest
```

Resolution Steps

1. Expand the PVC (CSI must support volume expansion):
 kubectl -n postgres-operator patch pvc <name> -p \
 '{"spec":{"resources":{"requests":{"storage":"500Gi"}}}}'
2. Wait for resize (may need pod restart for fs grow).
3. If WAL growing, check archive_command and pgBackRest:
 kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- \
 psql -c "SELECT pg_walfile_name(pg_current_wal_lsn());"
4. Reduce backup retention if appropriate:
 kubectl -n postgres-operator edit pg pgkubcluster
 # spec.backups.pgbackrest.repos[0].retention.full
5. VACUUM FULL on bloated tables (carefully, locks the table):
 pg_repack -k -t schema.table # online alternative

Prevention

- Monitor PV usage trend; alert at 70%, 80%, 90%

- Forecast growth and pre-provision +30% headroom
 - Schedule regular VACUUM ANALYZE
 - Use pgBackRest retention policy aligned with RPO
 - Use pgBackRest async archive-push
-

ALERT #29

CRITICAL

Pods, Jobs, Storage

KubePersistentVolumeErrors

Description

A PV is in Failed or Pending state.

Trigger Condition (PromQL)

```
kube_persistentvolume_status_phase{phase=~"Failed|Pending",job="kube-state-metrics"} > 0
```

Symptoms

- PVC stuck in Pending
- Pod stuck in ContainerCreating
- Cannot bring up new Postgres replica

Probable Root Causes

- csi-sscloud provisioner issues
- Quota exceeded on storage backend
- Reclaim policy mismatch
- Underlying storage backend (SAN/Ceph) error

Diagnostic Commands

```
kubectrl get pv | grep -v Bound
kubectrl describe pv <pv>

# Check CSI controller and node logs
kubectrl -n kube-system get pods -l app=csi-sscloud-controller
kubectrl -n kube-system logs csi-sscloud-controller-0 --all-containers --tail=200
kubectrl -n kube-system logs -l app=csi-sscloud-node --all-containers --tail=200
```

Resolution Steps

1. Check CSI controller logs for the actual error.
2. Verify storage backend health from outside Kubernetes.
3. If volume is orphaned, manually clean and recreate PVC.
4. For pending dynamic provisioning, check StorageClass config:
`kubectrl get sc`
5. Restart CSI controller if it's wedged:
`kubectrl -n kube-system rollout restart sts csi-sscloud-controller`

Prevention

- Monitor CSI driver health continuously
- Set storage quota with headroom
- Test PVC create/delete in non-prod regularly

ALERT #30

WARNING

Pods, Jobs, Storage

VolumeAttachment stuck

Description

Volume attachment has not completed in 10 minutes.

Trigger Condition (PromQL)

```
kube_volumeattachment_status_attached{job="kube-state-metrics"} == 0 and (time() - kube_volumeattachment_metadata_creation_timestamp) > 600
```

Symptoms

- Pod ContainerCreating with 'attach failed' events
- Postgres replica replacement stalls

Probable Root Causes

- Old attachment to a dead node not cleaned up (multi-attach)
- CSI node agent down on target node
- Underlying storage detach hang

Diagnostic Commands

```
kubectl get volumeattachment
kubectl describe volumeattachment <va-name>
kubectl -n kube-system logs -l app=csi-sscloud-node --tail=200
```

Resolution Steps

1. Confirm volume is **not** still attached to a dead node.
2. Force-delete the stuck VolumeAttachment (last resort):

```
kubectl patch volumeattachment <name> --type=json \
  -p='[{"op":"remove","path":"/metadata/finalizers"}]'
```
3. Bounce csi-sscloud-node DS **on** the target node:

```
kubectl -n kube-system delete pod -l app=csi-sscloud-node \
  --field-selector spec.nodeName=<node>
```

Prevention

- Use ReadWriteOnce only when truly needed; otherwise use RWX where possible
- Drain nodes properly before reboot to release attachments
- Ensure csi-sscloud-node tolerates node taints to detach during NotReady

3.5 Monitoring Stack

This category contains 4 alert(s). Each provides full diagnostic and remediation guidance.

ALERT #31

WARNING

Monitoring Stack

One or more targets are unreachable

Description

More than 10% of Prometheus targets in a service are down. Common in your env when nodes flap.

Trigger Condition (PromQL)

```
100 * (count(up == 0) BY (job, namespace, service) / count(up) BY (job, namespace, service)) > 10
```

Symptoms

- Gaps in Grafana dashboards
- Missing metrics for nodes/pods
- Compounded alerts firing because data is missing

Probable Root Causes

- Node down (see #9, #10)
- Service mismatch (Service selector and pod labels diverged)
- Network policy blocking Prometheus
- Authentication issue (kubelet metrics auth)
- Target endpoint /metrics returning 5xx

Diagnostic Commands

```
# Check Prometheus targets
kubectl -n monitoring port-forward svc/kube-prometheus-stack-prometheus 9090:9090
# Open http://localhost:9090/targets

# Check service endpoints
kubectl -n <ns> get endpoints <service>

# Try scraping directly
kubectl -n <ns> exec -it <prometheus-pod> -- \
  wget -qO- http://<target-ip>:<port>/metrics | head
```

Resolution Steps

1. Identify which targets are down from /targets endpoint.
2. If single target/job, fix that node/service.
3. If pattern (all kubelets), check kubelet --read-only-port or auth tokens.
4. Check NetworkPolicies in the namespace:
`kubectl -n <ns> get networkpolicy`

Prevention

- Use ServiceMonitor / PodMonitor with strict label selectors
- Monitor 'up' metric per scrape job
- Set Alertmanager group_wait/group_interval to dedupe noise from node flap

ALERT #32

CRITICAL

Monitoring Stack

Prometheus target absent (job_name absent)

Description

All targets for a critical job (e.g., kubelet, etcd, postgres-exporter) are missing.

Trigger Condition (PromQL)

```
absent(up{job="<critical-job>"}) == 1
```

Symptoms

- No metrics for the entire job tier

Probable Root Causes

- ServiceMonitor / PodMonitor deleted
- Selector mismatch
- RBAC removed Prometheus access

Diagnostic Commands

```
kubectl -n monitoring get servicemonitors
kubectl -n monitoring get podmonitors
kubectl -n monitoring describe prometheus kube-prometheus-stack
kubectl -n monitoring logs prometheus-kube-prometheus-stack-prometheus-0 --tail=200
```

Resolution Steps

1. Reapply the missing ServiceMonitor.
2. Verify the Prometheus CRD's `serviceMonitorSelector` matches.
3. Reconcile via [Helm](#): `helm upgrade kube-prometheus-stack prometheus-community/kube-prometheus-stack ...`

Prevention

- Manage monitoring stack via GitOps (ArgoCD)
- Use admission webhooks to prevent SM/PM deletions
- Test SM/PM application before promoting to prod

ALERT #33

WARNING

Monitoring Stack

PrometheusOperatorReconcileErrors

Description

Prometheus Operator failing to reconcile its CRDs.

Trigger Condition (PromQL)

```
rate(prometheus_operator_reconcile_errors_total{job="kube-prometheus-stack-operator"}[5m]) > 0.1
```

Symptoms

- New ServiceMonitors not being picked up
- Alertmanager config not updating

Probable Root Causes

- CRD validation error in a custom rule/SM
- Permission missing for operator SA
- Webhook failing

Diagnostic Commands

```
kubectl -n monitoring logs deploy/kube-prometheus-stack-operator --tail=200
kubectl -n monitoring get prometheusrule -A
kubectl -n monitoring describe prometheus kube-prometheus-stack-prometheus
```

Resolution Steps

1. Find the offending object in operator logs (object name will be in error).
2. Fix or remove the bad rule/ServiceMonitor.
3. Roll the operator to clear cached state:
`kubectl -n monitoring rollout restart deploy/kube-prometheus-stack-operator`

Prevention

- Validate PrometheusRule with promtool: `promtool check rules <file>`
- Use Renovate/Dependabot to keep operator version current

ALERT #34

CRITICAL

Monitoring Stack

AlertmanagerConfigInconsistent

Description

Alertmanager replicas are running with different configurations.

Trigger Condition (PromQL)

```
count by (cluster, namespace, service) (count_values by (cluster, namespace, service) ("config_hash", alertmanager_config_hash{job="kube-prometheus-stack-alertmanager"})) != 1
```

Symptoms

- Some alerts notify, others don't
- Inconsistent silencing
- Duplicate notifications

Probable Root Causes

- Stale config on one replica after restart
- Secret mismatch between replicas
- Race during config reload

Diagnostic Commands

```
kubectl -n monitoring get pods -l app.kubernetes.io/name=alertmanager
kubectl -n monitoring exec alertmanager-0 -- wget -qO- http://localhost:9093/-/reload
kubectl -n monitoring logs alertmanager-0 --tail=200
```

Resolution Steps

1. Force config **reload** across all replicas.
2. If still mismatched, **delete** the alertmanager-config secret **and** recreate via operator.
3. **Restart** the Alertmanager statefulset:
`kubectl -n monitoring rollout restart sts alertmanager-kube-prometheus-stack-alertmanager`

Prevention

- Manage AM config via AlertmanagerConfig CRDs
- Use sticky session / consistent hashing for AM gossip

3.6 PostgreSQL / Percona

This category contains 10 alert(s). Each provides full diagnostic and remediation guidance.

ALERT #35

CRITICAL

PostgreSQL / Percona

postgres backups failed

Description

pgBackRest backup did not complete successfully. From your env: pgbackupmonitor 29518550, 29519990, 29521430 all failed.

Trigger Condition (PromQL)

```
ccp_backrest_last_info_backrest_repo_backup_runtime_seconds == 0 or  
absent(ccp_backrest_last_info_backrest_repo_backup_runtime_seconds)
```

Symptoms

- Stale 'last successful backup' metric
- RPO at risk
- PITR window shrinking

Probable Root Causes

- pgBackRest cannot reach repo storage
- Repo full
- Permissions / SELinux / fsGroup issue
- Stanza upgrade pending after PG version change
- WAL archive backlog blocking new backups
- Network policy blocking ports

Diagnostic Commands

```
# Backup history and status  
kubectl -n postgres-operator exec pgkubcluster-repo-host-0 -- pgbackrest info  
  
# pgBackRest log  
kubectl -n postgres-operator exec pgkubcluster-repo-host-0 -- \  
tail -200 /pgbackrest/log/db-backup.log  
  
# Stanza check  
kubectl -n postgres-operator exec pgkubcluster-repo-host-0 -- pgbackrest --stanza=db  
check  
  
# Archive status from primary  
kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- \  
psql -c "SELECT * FROM pg_stat_archiver;"  
  
# Operator backup CR status  
kubectl -n postgres-operator get pgbackup,pgbackrestbackup 2>/dev/null
```


Resolution Steps

1. Read `pgBackRest` log for the specific failure code (e.g., `ERROR: [055], [087]`).
2. If repo unreachable, check repo-host pod and network:
`kubectl -n postgres-operator describe pod pgkubcluster-repo-host-0`
3. If repo full, free space:
`pgbackrest --stanza=db expire # honors retention`
4. After PG version bump, run stanza-upgrade:
`kubectl -n postgres-operator exec pgkubcluster-repo-host-0 -- \`
`pgbackrest --stanza=db stanza-upgrade`
5. Trigger a manual backup once root cause is fixed:
`kubectl annotate -n postgres-operator pg pgkubcluster \`
`postgres-operator.crunchydata.com/pgbackrest-backup="manual-$(date +%s)"`
`--overwrite`
6. Validate backup is restorable: do a test restore in lower environment.

Prevention

- Monitor `ccp_backrest_last_info_backrest_repo_backup_runtime_seconds`
 - Alert at 1.5× of expected backup window
 - Test restore monthly in non-prod from latest backup
 - Configure 2 repos (local + remote) for redundancy
 - Keep `pgBackRest` log centralized (Loki/ELK)
 - Run `pgbackrest verify` weekly
-

ALERT #36

WARNING

PostgreSQL / Percona

PostgreSQLReplicationLag

Description

Replica lag exceeds 16 MB or 30+ seconds behind primary.

Trigger Condition (PromQL)

```
ccp_replication_lag_size_bytes > 16777216 # 16MB
```

Symptoms

- Read queries on replica returning stale data
- Patroni shows lag column non-zero
- Failover RPO at risk

Probable Root Causes

- Network bandwidth saturation between primary and replica
- Replica overloaded by reporting workloads
- Long transaction blocking apply on replica (hot_standby_feedback)
- WAL replay slower than generation
- Replication slot stuck

Diagnostic Commands

```
# Patroni view
kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- patronictl list

# From primary: replication slots and lag
kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- psql -c \
  "SELECT client_addr, state, sent_lsn, write_lsn, flush_lsn, replay_lsn, \
    pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS lag \
  FROM pg_stat_replication;"

# Long-running queries on replica
kubectl -n postgres-operator exec <replica-pod> -c database -- psql -c \
  "SELECT pid, now()-query_start AS duration, query \
  FROM pg_stat_activity WHERE state='active' ORDER BY duration DESC;"
```

Resolution Steps

1. Cancel long-running queries on the replica (if blocking apply).
2. Check network: iperf3 between primary and replica pods.
3. If slot stuck, drop and recreate (will require replica re-init):
`SELECT pg_drop_replication_slot('<slot>');`
4. Tune wal_receiver_timeout, max_wal_senders.
5. Patroni reinit if hopelessly behind:
`patronictl reinit pgkubcluster <member>`

Prevention

- Separate OLTP and reporting workloads (use a dedicated reporting replica)
- Provision adequate network bandwidth between primary and replicas
- Tune max_wal_size and checkpoint_timeout for steady WAL flow
- Monitor pg_replication_slots.active and confirmed_flush_lsn

ALERT #37

WARNING

PostgreSQL / Percona

PostgreSQLConnectionsTooHigh

Description

Active connections exceed 85% of max_connections.

Trigger Condition (PromQL)

```
ccp_connection_stats_total / ccp_connection_stats_max_connections > 0.85
```

Symptoms

- FATAL: too many connections for role / database
- App connection timeouts
- PgBouncer queueing

Probable Root Causes

- Connection leak in app (forgot to close)
- PgBouncer pool size too high or bypassed
- max_connections too low
- Burst traffic

Diagnostic Commands

```
kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- psql -c \  
"SELECT count(*), state, application_name FROM pg_stat_activity \  
GROUP BY state, application_name ORDER BY count(*) DESC;"  
  
# Long idle-in-transaction (worst case)  
kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- psql -c \  
"SELECT pid, username, application_name, now()-state_change AS idle_for \  
FROM pg_stat_activity WHERE state='idle in transaction' \  
ORDER BY idle_for DESC LIMIT 20;"  
  
# PgBouncer stats  
kubectl -n postgres-operator exec pgkubcluster-pgbouncer-* -- \  
psql -h 127.0.0.1 -p 5432 -U pgbouncer pgbouncer -c "SHOW POOLS;"
```

Resolution Steps

1. Kill idle-in-transaction sessions older than X minutes:
`SELECT pg_terminate_backend(pid) FROM pg_stat_activity \
WHERE state='idle in transaction' AND now()-state_change > interval '10 minutes';`
2. Identify the leaking app (group by application_name) and fix it.
3. Raise max_connections (requires restart) only if PgBouncer not feasible.
4. Tune PgBouncer pool_size and max_client_conn.
5. Switch app to use PgBouncer (transaction mode) if not already.

Prevention

- Always front Postgres with PgBouncer (already deployed in your env)
- Set idle_in_transaction_session_timeout = '5min'
- Set statement_timeout for OLTP
- Monitor pg_stat_activity counts in Grafana
- Use connection pooling in apps (HikariCP, pgx, etc.)

ALERT #38

WARNING

PostgreSQL / Percona

PostgreSQLDeadlocks

Description

Database is reporting deadlocks.

Trigger Condition (PromQL)

```
rate(ccp_stat_database_deadlocks[5m]) > 0
```

Symptoms

- App errors: 'deadlock detected'
- Transactions abort and retry

Probable Root Causes

- Concurrent access to same rows in different orders
- Missing index causing wide locks
- Long-running transactions

Diagnostic Commands

```
kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- psql -c \  
"SELECT datname, deadlocks FROM pg_stat_database ORDER BY deadlocks DESC;"  
  
# Postgres logs (if log_lock_waits=on)  
kubectl -n postgres-operator logs pgkubcluster-i1-rvcz-0 -c database | grep -i deadlock
```

Resolution Steps

1. Enable log_lock_waits and deadlock_timeout = 1s to capture details.
2. From log entries, identify the two queries and tables involved.
3. Modify app code to acquire locks in consistent order.
4. Add appropriate indexes to narrow lock scope.

Prevention

- Enforce consistent locking order in app code
- Keep transactions short
- Add indexes for FK columns
- Use SELECT ... FOR UPDATE NOWAIT / SKIP LOCKED where appropriate

ALERT #39

CRITICAL

PostgreSQL / Percona

PostgreSQL down / not responding

Description

No primary or no Postgres pod responding to health checks.

Trigger Condition (PromQL)

```
ccp_is_in_recovery == 1 and ccp_postgresql_version{} == 0 # primary missing
```

Symptoms

- App cannot connect to DB
- Patroni shows no leader
- Replicas all in standby with no replication

Probable Root Causes

- Primary pod crashed and replica failover not happening
- Patroni DCS (etcd) lost
- All replicas behind WAL gap, can't promote
- PVC issue on primary

Diagnostic Commands

```
kubectl -n postgres-operator get pods -l postgres-operator.crunchydata.com/cluster=pgkub
cluster

kubectl -n postgres-operator exec <any-pg-pod> -c database -- patronictl list

kubectl -n postgres-operator logs <pod> -c database --tail=200
kubectl -n postgres-operator logs <pod> -c patroni --tail=200
```

Resolution Steps

1. If pod crashed but PVC ok, **kill** the pod to let sts recreate:
`kubectl -n postgres-operator delete pod <primary-pod>`
2. If Patroni cannot **find** leader, manually fail over:
`patronictl failover --candidate <healthy-replica>`
3. If WAL gap, restore from **pgBackRest**:
`pgbackrest --stanza=db restore --type=time --target="YYYY-MM-DD HH:MM:SS"`
4. If DCS lost, check **etcd** (Alerts 1-5).

Prevention

- Run 3+ Postgres replicas across nodes
- Use pod anti-affinity to spread replicas
- Test failover quarterly
- Maintain pgBackRest with differential and WAL archiving
- Monitor patroni_xlog_replayed_location lag

ALERT #40

WARNING

PostgreSQL / Percona

PostgreSQL Slow Queries / High Latency

Description

P99 query latency exceeds 5 seconds.

Trigger Condition (PromQL)

```
histogram_quantile(0.99, rate(ccp_query_duration_seconds_bucket[5m])) > 5
```

Symptoms

- App user-facing latency
- High DB CPU

Probable Root Causes

- Missing or unused index
- Stale statistics (need ANALYZE)
- Bloated table
- Bad plan due to parameter sniffing
- I/O saturation

Diagnostic Commands

```
# Top queries by total time
kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- psql -c \
  "SELECT query, calls, total_exec_time, mean_exec_time, rows \
  FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 20;"

# Tables needing vacuum
kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- psql -c \
  "SELECT relname, n_dead_tup, last_autovacuum FROM pg_stat_user_tables \
  ORDER BY n_dead_tup DESC LIMIT 10;"

# EXPLAIN a slow query
kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- psql -c \
  "EXPLAIN (ANALYZE, BUFFERS) <query>;"
```

Resolution Steps

1. Identify top 5 queries from pg_stat_statements.
2. EXPLAIN ANALYZE each, look for Seq Scan on large tables.
3. Add indexes; CREATE INDEX CONCURRENTLY to avoid locks.
4. Run VACUUM ANALYZE on tables with stale stats.
5. For repeated bloat, schedule pg_repack.
6. Increase work_mem if many sorts spilling to disk.

Prevention

- Enable pg_stat_statements (already part of Percona images)
- Schedule weekly pg_stat_statements review
- Run ANALYZE after large data loads
- Monitor cache hit ratio (should be >95%)
- Use auto_explain for slow query logging

ALERT #41

WARNING

PostgreSQL / Percona

PgBouncer connections high / pool saturated

Description

PgBouncer pool is over 85% used.

Trigger Condition (PromQL)

```
pgbouncer_pools_server_active_connections / pgbouncer_pools_server_max_connections > 0.85
```

Symptoms

- Client connections wait in `cl_waiting`
- App p99 latency spike

Probable Root Causes

- `pool_size` too small
- Long transactions holding server connection
- App not using transaction mode efficiently

Diagnostic Commands

```
kubectl -n postgres-operator exec pgkubcluster-pgbouncer-<id> -- \
psql -h 127.0.0.1 -p 5432 -U pgbouncer pgbouncer -c "SHOW POOLS;"

kubectl -n postgres-operator exec pgkubcluster-pgbouncer-<id> -- \
psql -h 127.0.0.1 -p 5432 -U pgbouncer pgbouncer -c "SHOW CLIENTS;"

kubectl -n postgres-operator exec pgkubcluster-pgbouncer-<id> -- \
psql -h 127.0.0.1 -p 5432 -U pgbouncer pgbouncer -c "SHOW STATS;"
```

Resolution Steps

1. Increase `pool_size` in `PerconaPGCluster spec.proxy.pgBouncer.config`.
2. Find sessions holding server conns `>5` min and kill via Postgres.
3. Switch to transaction pooling mode if currently session mode.
4. Scale PgBouncer replicas (already 2/2 in your env).

Prevention

- Right-size pool: `target_pool_size = (db_max_connections - reserved) / pgbouncer_replicas`
- Use transaction mode unless app uses session features (LISTEN/NOTIFY, prepared statements with names)
- Monitor `cl_waiting > 0` as canary

ALERT #42

CRITICAL

PostgreSQL / Percona

PgBouncer Down

Description

All PgBouncer pods are unreachable. From your env: pgkubcluster-pgbouncer-b7f656889-k4j7c and -s62tn.

Trigger Condition (PromQL)

```
absent(up{job="pgbouncer"} == 1) for 5m
```

Symptoms

- App cannot connect to DB (apps connect via PgBouncer)
- Service endpoints empty

Probable Root Causes

- Both PgBouncer pods crashed simultaneously
- Service selector mismatch
- TLS cert expired

Diagnostic Commands

```
kubectl -n postgres-operator get pods -l postgres-operator.crunchydata.com/role=pgbouncer
kubectl -n postgres-operator describe pod pgkubcluster-pgbouncer-<id>
kubectl -n postgres-operator logs pgkubcluster-pgbouncer-<id>
```

Resolution Steps

1. Bounce stuck PgBouncer pods:

```
kubectl -n postgres-operator delete pod -l postgres-operator.crunchydata.com/role=pgbouncer
```
2. Verify TLS certs are **not** expired in the secret:

```
kubectl -n postgres-operator get secret pgkubcluster-pgbouncer -o yaml
```
3. Reconcile via operator: bump replicas in CR **or** annotate to force re-reconcile.

Prevention

- Run ≥ 2 PgBouncer replicas with anti-affinity (already in your env)
- Configure PgBouncer liveness probes correctly
- Auto-renew TLS certs via cert-manager

ALERT #43

WARNING

PostgreSQL / Percona

Percona Operator reconcile failed

Description

Operator failing to reconcile a PerconaPGCluster CR.

Trigger Condition (PromQL)

```
rate(controller_runtime_reconcile_errors_total{controller="perconapgcluster"}[5m]) > 0
```

Symptoms

- Spec changes not taking effect
- Status conditions show errors

Probable Root Causes

- Invalid CR spec
- RBAC error
- Webhook timeout
- API server overloaded

Diagnostic Commands

```
kubectl -n postgres-operator logs deploy/percona-postgresql-operator --tail=500 | grep -i error
kubectl -n postgres-operator get pg pgkubcluster -o yaml | yq '.status'
kubectl -n postgres-operator describe pg pgkubcluster
```

Resolution Steps

1. Look at operator logs **for** the specific reconcile error.
2. Validate the CR with **kubectl** `apply --dry-run=server`.
3. **If** RBAC, ensure the operator's **ClusterRole** has the required permissions.
4. **Restart** the operator **if** it appears wedged:
`kubectl -n postgres-operator rollout restart deploy/percona-postgresql-operator`

Prevention

- Manage CRs via GitOps (ArgoCD)
- Run operator in HA mode (>=2 replicas with leader election)
- Subscribe to Percona Operator releases for known-issue awareness

ALERT #44

CRITICAL

PostgreSQL / Percona

Last successful backup is older than RPO

Description

No successful pgBackRest backup in 24 hours.

Trigger Condition (PromQL)

```
(time() - ccp_backrest_last_info_backrest_repo_backup_complete) > 86400 # >24h
```

Symptoms

- RPO breach
- WAL archive may be only recovery option

Probable Root Causes

- Same as Alert #35 (postgres backups failed) but persistent

Diagnostic Commands

```
kubectl -n postgres-operator exec pgkubcluster-repo-host-0 -- pgbackrest info
kubectl -n postgres-operator exec pgkubcluster-repo-host-0 -- \
  bash -c 'find /pgbackrest -name "*.history" -mtime -7 -ls | tail -5'
```

Resolution Steps

```
1. Resolve underlying backup failure (Alert #35).
2. Trigger backup immediately:
  kubectl annotate -n postgres-operator pg pgkubcluster \
    postgres-operator.crunchydata.com/pgbackrest-backup="manual-$(date +%s)"
  --overwrite
3. Confirm new backup landed:
  pgbackrest --stanza=db info
```

Prevention

- Daily backup with full weekly + diff/incr daily
- Alert on backup-age > RPO immediately, not after backlog
- Off-cluster backup copy (S3 or remote NAS)
- Document RTO/RPO and test against backup chain

3.7 Network & Kubelet

This category contains 6 alert(s). Each provides full diagnostic and remediation guidance.

ALERT #45

WARNING

Network & Kubelet

KubeAPILatencyHigh

Description

P99 API server request latency > 4 seconds.

Trigger Condition (PromQL)

```
histogram_quantile(0.99, sum by (verb, le) (rate(apiserver_request_duration_seconds_bucket{verb!~"WATCH|CONNECT"}[5m]))) > 4
```

Symptoms

- kubectl feels slow
- Operator reconcile loops timeout
- Webhooks fail

Probable Root Causes

- etcd slow (Alerts 1-5)
- API server CPU/Mem starvation
- Heavy LIST requests (kubectl get pods -A in big cluster)
- Webhook timeouts

Diagnostic Commands

```
kubectl -n kube-system top pods -l component=kube-apiserver

# Slowest verbs
# Prom: histogram_quantile(0.99, sum by (verb, le)
(rate(apiserver_request_duration_seconds_bucket[5m])))

# Audit log for heavy clients
ssh <cp-node> sudo grep 'list' /var/log/kube-apiserver-audit.log | wc -l
```

Resolution Steps

1. Identify slow verb (LIST, GET, etc.).
2. If LIST: find the noisy client; switch to watch+informer.
3. Increase apiserver --max-requests-inflight if CPU available.
4. Tune etcd if backend slow.
5. Move heavy webhooks (e.g., OPA) off-critical path.

Prevention

- Enforce informer-based access patterns in operators/controllers
- Use serverSideApply where possible
- Set reasonable webhook timeouts (5s)
- Run apiserver with adequate CPU

ALERT #46

WARNING

Network & Kubelet

Kubernetes Client Certificate Expiration

Description

Client certificates used by kubelets/components expire within 7 days.

Trigger Condition (PromQL)

```
apiserver_client_certificate_expiration_seconds_count{job="apiserver"} > 0 and on(job) histogram_quantile(0.01, sum by (job, le) (rate(apiserver_client_certificate_expiration_seconds_bucket{job="apiserver"}[5m]))) < 86400 * 7
```

Symptoms

- kubelets will lose ability to authenticate
- Cluster slowly degrading after expiration

Probable Root Causes

- Certs not auto-renewed
- kubeadm renewal cron not run

Diagnostic Commands

```
ssh <cp-node> sudo kubeadm certs check-expiration

# Per-component cert
ssh <cp-node> sudo openssl x509 -in /etc/kubernetes/pki/apiserver.crt -noout -enddate
```

Resolution Steps

1. Renew on each control-plane node:
`sudo kubeadm certs renew all`
2. Restart static pods (apiserver, controller-manager, scheduler):
`sudo crictl ps -a --name kube-apiserver -q | xargs sudo crictl rm -f`
3. Distribute updated kubeconfigs to ops/admins.
4. Restart kubelet if its cert was renewed: `systemctl restart kubelet`

Prevention

- Enable kubelet rotateCertificates: true and serverTLSBootstrap: true
- Cron job: monthly 'kubeadm certs renew all'
- Monitor cert expiry in Prometheus alert at 30/14/7 days

ALERT #47

CRITICAL

Network & Kubelet

Cilium agent down on node

Description

Cilium agent (CNI) not running on at least one node.

Trigger Condition (PromQL)

```
absent_over_time(cilium_agent_up[5m]) == 1
```

Symptoms

- New pods on the node fail to get networking
- Existing pod connectivity may degrade
- NetworkPolicy not enforced

Probable Root Causes

- Pod crashed (eBPF map failure, kernel mismatch)
- Node disk pressure prevented restart
- Cilium DS rollout stuck

Diagnostic Commands

```
kubectl -n kube-system get pods -l k8s-app=cilium -o wide
kubectl -n kube-system logs -l k8s-app=cilium --tail=200

# From a healthy cilium pod, status
kubectl -n kube-system exec ds/cilium -- cilium status --brief
```

Resolution Steps

1. Restart the agent on the affected node:
`kubectl -n kube-system delete pod -l k8s-app=cilium \`
`--field-selector spec.nodeName=<node>`
2. If kernel module issue, check kernel version:
`ssh <node> uname -r`
3. Check eBPF map size limits:
`ssh <node> bpftool map show`
4. Drain and replace node if persistently failing.

Prevention

- Pin Cilium version; do not auto-upgrade
- Validate kernel ≥ 5.4 for new Cilium features
- Test Cilium upgrades in beta cluster first
- Use Hubble to validate connectivity post-deploy

ALERT #48

WARNING

Network & Kubelet

CoreDNS Down or High Error Rate

Description

CoreDNS error rate >1% or pods unavailable.

Trigger Condition (PromQL)

```
(rate(coredns_dns_responses_total{rcode="SERVFAIL"}[5m]) /  
rate(coredns_dns_responses_total[5m])) > 0.01
```

Symptoms

- Pods fail name resolution intermittently
- Postgres replica cannot reach primary by service name
- Operator API calls fail intermittently

Probable Root Causes

- Upstream DNS unreachable
- CoreDNS pod crash
- Cache memory pressure
- Misconfigured forward zone

Diagnostic Commands

```
kubectl -n kube-system get pods -l k8s-app=kube-dns  
kubectl -n kube-system logs -l k8s-app=kube-dns --tail=200  
  
# Test DNS from a debug pod  
kubectl run -it --rm dns-test --image=busybox:1.36 --restart=Never -- \\\  
  nslookup pgkubcluster-primary.postgres-operator.svc.cluster.local  
  
# CoreDNS configmap  
kubectl -n kube-system get cm coredns -o yaml
```

Resolution Steps

1. Bounce CoreDNS pods: `kubectl -n kube-system rollout restart deploy/coredns`
2. Ensure 2+ replicas with anti-affinity.
3. Verify upstream resolvers reachable from nodes.
4. Increase CoreDNS memory limit if cache full.
5. Add cache plugin tuning: cache 30 (TTL).

Prevention

- Run CoreDNS as Deployment with HPA
- Use NodeLocal DNSCache for hot path
- Monitor `coredns_panic_count_total`

ALERT #49

CRITICAL

Network & Kubelet

KubeletDown

Description

Kubelet unreachable from Prometheus on one or more nodes.

Trigger Condition (PromQL)

```
absent(up{job="kubelet", metrics_path="/metrics"} == 1)
```

Symptoms

- Node may go NotReady
- Pod metrics missing from that node

Probable Root Causes

- Kubelet stopped/crashed
- Cert rotation issue
- Read-only port disabled but auth misconfigured

Diagnostic Commands

```
ssh <node> systemctl status kubelet
ssh <node> journalctl -u kubelet --since '15 min ago' --no-pager | tail -50

# Test from another node
curl -k https://<node-ip>:10250/healthz
```

Resolution Steps

1. Restart kubelet: `ssh <node> sudo systemctl restart kubelet`
2. If cert issue, regenerate `/var/lib/kubelet/pki` and restart.
3. Validate kubelet config: `/var/lib/kubelet/config.yaml`

Prevention

- Enable kubelet rotateCertificates
- Monitor systemd unit status via node_exporter
- Pin kubelet version across nodes

ALERT #50

CRITICAL

Network & Kubelet

KubeProxyDown

Description

kube-proxy not running on one or more nodes.

Trigger Condition (PromQL)

```
absent(up{job="kube-proxy"} == 1)
```

Symptoms

- Service ClusterIPs not reachable from that node
- App connections to services fail intermittently

Probable Root Causes

- kube-proxy pod crashed
- iptables / IPVS module missing
- Configmap misconfigured

Diagnostic Commands

```
kubectl -n kube-system get pods -l k8s-app=kube-proxy -o wide
kubectl -n kube-system logs -l k8s-app=kube-proxy --tail=200

# Test service from node
ssh <node> curl -v http://10.96.0.1:443 -k
```

Resolution Steps

1. Bounce kube-proxy pod on the affected node:
`kubectl -n kube-system delete pod -l k8s-app=kube-proxy \`
`--field-selector spec.nodeName=<node>`
2. Verify ipvsadm/iptables work on the node:
`ssh <node> sudo iptables -t nat -L | head`
3. Note: in your env Cilium may replace kube-proxy via 'kube-proxy replacement'.
Verify which is authoritative: `kubectl -n kube-system describe ds kube-proxy`

Prevention

- Standardize on either kube-proxy OR Cilium kube-proxy-replacement, not both
- Pin kube-proxy version to match Kubernetes minor version
- Health-check service connectivity from each node periodically

Appendix A — Operator Cheat Sheet

Common kubectl Commands

```
# Cluster-wide view
kubectl get nodes -o wide
kubectl get pods -A -o wide
kubectl get events -A --sort-by='.lastTimestamp' | tail -50

# PerconaPGCluster status
kubectl -n postgres-operator get pg
kubectl -n postgres-operator describe pg pgkubcluster
kubectl -n postgres-operator get pods -l postgres-operator.crunchydata.com/cluster=pgkubcluster

# Patroni
kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- patronictl list
kubectl -n postgres-operator exec pgkubcluster-i1-rvcz-0 -c database -- patronictl history

# pgBackRest
kubectl -n postgres-operator exec pgkubcluster-repo-host-0 -- pgbackrest info
kubectl -n postgres-operator exec pgkubcluster-repo-host-0 -- pgbackrest --stanza=db check

# PgBouncer
kubectl -n postgres-operator exec deploy/pgkubcluster-pgbouncer -- \
  psql -h 127.0.0.1 -p 5432 -U pgbouncer pgbouncer -c 'SHOW POOLS;'
```

Common psql Diagnostics

```
-- Active sessions
SELECT pid, username, application_name, state, now()-query_start AS duration, query
FROM pg_stat_activity
WHERE state != 'idle' ORDER BY duration DESC NULLS LAST;

-- Replication status (run on primary)
SELECT client_addr, state, sent_lsn, write_lsn, replay_lsn,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS lag
FROM pg_stat_replication;

-- Cache hit ratio (target >95%)
SELECT sum(blks_hit)*100.0/sum(blks_hit+blks_read) AS cache_hit_ratio
FROM pg_stat_database;

-- Top expensive queries
SELECT query, calls, total_exec_time, mean_exec_time, rows
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 20;

-- Bloat / dead tuples
SELECT relname, n_dead_tup, last_autovacuum
FROM pg_stat_user_tables ORDER BY n_dead_tup DESC LIMIT 20;

-- Long idle-in-transaction sessions
SELECT pid, username, application_name, now()-state_change AS idle_for
FROM pg_stat_activity
WHERE state='idle in transaction' ORDER BY idle_for DESC;

-- Cancel / terminate a session
SELECT pg_cancel_backend(<pid>);
SELECT pg_terminate_backend(<pid>);
```

Critical First-Response Commands

```
# 'Is the cluster up?' triage
kubectl get cs # legacy but useful
kubectl get nodes
kubectl -n kube-system get pods

# 'Is etcd healthy?'
kubectl -n kube-system exec etcd-<cp-node> -- sh -c \
'ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key endpoint health --cluster -w table'

# 'Are backups recent?'
kubectl -n postgres-operator exec pgkubcluster-repo-host-0 -- pgbackrest info

# 'What changed?'
kubectl get events -A --sort-by='.lastTimestamp' | tail -30
```

Appendix B — Runbook Conventions

Naming Conventions Used Above

- <cp-node>: any control-plane node hostname
- <node>: any worker node hostname
- <pod>: a specific pod name (substitute as appropriate)
- pgkubcluster: example PerconaPGCluster CR name (Cluster A)
- salt-control-postgres: example PerconaPGCluster CR name (Cluster B)
- i1, i2, i3: instance set identifiers under a PerconaPGCluster

On-Call Escalation

For any **CRITICAL** alert that is not resolved within 30 minutes of being acknowledged, escalate to the next tier per the team's escalation matrix. Document timeline and actions in the incident ticket; attach diagnostic outputs.